FE

MGU BCA - 3RD SEM

Feature Engineering

Complete Important Questions

By Aben Sojan

Exam - 18 Nov 2026

MODULE 1

Introduction and Preprocessing

1

EXAM PATTERN (from real MGU question papers)

FE theory exam: 35 marks, 1 hour (MG3DSEBCA201 Honours model paper, mgu.ac.in Oct 2025).

Printed: 10 x Part A (2 marks) + 5 x Part B (5 marks) = 45 marks printed, so there IS choice.

Likely: Part A all compulsory (20) + answer any 3 of 5 in Part B (15) = 35. Exact choice rule UNVERIFIED - confirm from the paper.

Part A questions carry Bloom tags R/U/An and CO mappings in the paper.

Module 1 (data types, missing data, cleaning, scaling) gave 6 of the 10 Part A questions in the model paper - it is the Part A bank.

Every module, every important, repeated and sure-mark question for Feature Engineering, each with a full-mark model answer.

1 What is a feature in machine learning? Give examples.

IMPORTANT

Definition: A feature is a single measurable property of the data that is given as input to a machine learning model.

In a dataset, each column is usually one feature and each row is one record or example.

The model learns patterns only from the features it is given, so the choice of features decides what the model can learn.

Examples: in a house-price dataset, area in square feet, number of bedrooms and location are features.

In a student dataset, age, attendance percentage and marks are features.

In a spam filter, features can be the number of links in the mail or the presence of words like lottery.

Features can be numerical (age, salary) or categorical (city, gender).

Raw columns are sometimes not enough, so new features are created from old ones; this is called feature engineering.

Example: from a date of birth column we can create an age feature that the model understands better.

In short, features are the language through which data speaks to the model, and good features lead to good predictions.

2 What is data? How do features relate to a dataset?

IMPORTANT

Definition: Data is the raw collection of facts and figures gathered from the real world, like student marks or house prices.

A dataset organises this data as a table: each row is one record and each column is one feature.

So features are simply the columns of the dataset that describe each record.

Example: a student table has rows for each student and columns like name, age and marks.

Here age and marks are the features, because they are the measurable inputs the model will learn from.

Name is also a column, but it is usually not a useful feature since it carries no pattern.

The target column, like final result pass or fail, is what the model tries to predict and is not counted as a feature.

In short, data is the whole table, and features are the meaningful columns of that table that drive learning.

3 Differentiate between data and information.

IMPORTANT

Definition: Data is raw unprocessed facts and figures, while information is data that has been processed into something meaningful.

Data on its own answers nothing: a list of marks 45, 67, 89 tells you nothing until you know whose marks they are.

Information adds context and purpose: the class average is 67 and Aben scored 89, the highest in class.

Data is the input; information is the output of processing, cleaning and analysing that data.

Example: daily temperature readings for a year are data; the conclusion that July is the hottest month is information.

In machine learning, raw datasets are data, and the patterns the model learns from them become information.

One more difference: the same data can produce different information depending on the question asked.

In short, data is raw material and information is the finished product made from it.

4 Why are features important in machine learning?

IMPORTANT

A machine learning model sees only the features, so the quality of features decides the quality of learning.

If the features are weak or irrelevant, even the best algorithm will give poor predictions.

Example: predicting house price without the location feature gives bad results, because location strongly affects price.

Good features improve accuracy without needing a bigger, slower or costlier model.

They also reduce training time, because the model finds patterns faster in clean, meaningful inputs.

Well-chosen features reduce overfitting, since the model does not get distracted by noise.

In real projects, improving features usually gives a bigger accuracy jump than tuning the algorithm.

Hence feature engineering is often called the most important step of the machine learning pipeline.

5 'Feature engineering matters more than the algorithm.' Explain.

IMPORTANT

This statement means that the information fed into a model matters more than the model itself.

A simple model with great features often beats a complex model with poor features.

Example: adding a days since last purchase feature improves a sales model more than switching algorithms.

The algorithm can only combine and weight what it is given; it cannot invent missing information.

If a key signal like customer location is absent, no algorithm can recover it.

On the other hand, with the right features, even basic models like linear regression perform very well.

That is why experienced data scientists spend most of their time on features, not on model tuning.

In short, features decide the ceiling of performance, while the algorithm only decides how close we get to it.

6 Distinguish between discrete and continuous data with examples.

IMPORTANT

Definition: Discrete data takes separate, countable values, while continuous data can take any value in a range.

Discrete data is counted in whole numbers: number of students in a class, number of children in a family, number of cars sold.

It has clear gaps between values; you cannot have 2.5 children.

Continuous data is measured and can take decimal values: height 172.5 cm, weight 54.2 kg, time 9.58 seconds.

Between any two continuous values there is always another possible value.

Example: the number of books on a shelf is discrete, but the time taken to read one book is continuous.

This difference matters because some models and plots suit one type better than the other.

In short, countable values are discrete and measurable values are continuous.

7 Explain nominal and ordinal data with examples.

IMPORTANT

Both nominal and ordinal are types of categorical (label) data, but they differ in order.

Nominal data has categories with no natural order: gender, city, colour, blood group.

You can count how many fall in each category, but you cannot rank them.

Ordinal data has categories with a clear meaningful order: grades A, B, C, D or ranks 1st, 2nd, 3rd.

Example: shirt sizes S, M, L and XL are ordinal because M is clearly larger than S.

The order in ordinal data carries information, while in nominal data it does not.

This matters in encoding: ordinal data can use label encoding that respects order, nominal data usually needs one-hot encoding.

In short, nominal labels only name groups, while ordinal labels also rank them.

8 What is the difference between interval and ratio data?

IMPORTANT

Both interval and ratio are numerical data types, and the difference lies in the meaning of zero.

Interval data has equal gaps between values but no true zero: temperature in Celsius is the classic example.

0 degree Celsius does not mean no heat, it is just a point on the scale.

Ratio data has a true zero that means the complete absence of the quantity: weight, height, age, income.

Because of the true zero, ratios are meaningful for ratio data: 20 kg is truly twice as heavy as 10 kg.

But for interval data ratios mislead: 20 C is not twice as hot as 10 C.

Both types support addition and subtraction, but only ratio data supports meaningful multiplication and division.

In short, the true zero is what separates ratio data from interval data.

9 Why must categorical data be converted before using ML models?

IMPORTANT

Machine learning models do mathematics, and mathematics needs numbers, not words.

A model cannot multiply or compare the word Red, so every category must become a number first.

Encoding methods like one-hot encoding and label encoding perform this conversion.

Without conversion, most algorithms simply fail to run or silently misbehave on text columns.

Even models that accept categories internally still convert them to numbers behind the scenes.

The choice of encoding also matters: a wrong encoding can create false order between categories.

Example: encoding cities as 1, 2, 3 may trick a linear model into thinking city 3 is greater than city 1.

Hence converting categorical data correctly is a compulsory step before training.

10 What is the significance of ordinal data in decision making?

IMPORTANT

Ordinal data is categorical data with a meaningful order, like grades A, B, C or satisfaction levels.

Its significance is that the order itself carries information that pure labels do not.

Example: knowing a customer rated service as Poor, Average or Excellent lets a manager rank and prioritise complaints.

A decision maker can set thresholds on ordinal data: all feedback below Good needs attention.

Unlike nominal data, ordinal data supports comparisons like better than or worse than.

Unlike full numbers, it does not support exact maths: the gap between Good and Excellent is not measurable.

In practice it is often converted to ordered numbers (1 to 5) for analysis, as long as the order is respected.

In short, ordinal data matters because its built-in ranking directly guides prioritised decisions.

11 What is missing data? How is it handled?

IMPORTANT

Definition: Missing data means some values are absent in the dataset, shown as blank cells or NaN.

It happens due to skipped survey answers, sensor failures or errors while merging files.

Missing data must never be ignored silently, because most models cannot train on empty cells and the gaps can bias results.

Handling method 1: fill small gaps using imputation, replacing blanks with the mean, median or mode of the column.

Handling method 2: fill with a sensible constant, like 0 for number of children or Unknown for a city.

Handling method 3: drop the row or column, but only when very little data is affected or the column is mostly empty.

The right choice depends on how much is missing and why it is missing.

In short, detect the gaps first, then fill them sensibly or drop them carefully, and always document what was done.

12 Explain mean, median and mode imputation with examples.

IMPORTANT

Imputation means filling missing values with a sensible replacement instead of deleting the row.

Mean imputation replaces blanks with the average of the column; it suits normal numeric data without outliers.

Example: ages 20, 22, 24 with one missing become 20, 22, 24, 22, where 22 is the mean.

Median imputation replaces blanks with the middle value; it suits skewed data because outliers do not pull it.

Example: salaries 30000, 32000, 5000000 with one missing get the median 32000, not the distorted mean.

Mode imputation replaces blanks with the most frequent value; it is used for categorical columns.

Example: a city column missing in a few rows gets Mumbai if Mumbai appears most often.

In short: mean for normal numbers, median for skewed numbers, mode for categories.

13 When should a column with missing values be dropped instead of filled?

IMPORTANT

Filling is the default choice when only a small part of the column is missing, because the column still carries useful information.

A column should be dropped when most of its values are missing, commonly more than 50 to 60 percent.

Filling such a column would mean inventing more data than actually exists, which misleads the model.

Example: a middle name column missing in 80 percent of rows is better dropped than filled.

Similarly, drop a row only when it has too many missing values and very few rows are affected overall.

Dropping too many rows shrinks the dataset and can remove valuable patterns.

The decision should also consider importance: a highly predictive column deserves careful filling, not quick dropping.

In short, fill small gaps, drop columns that are mostly empty, and never drop blindly.

14 What is data cleaning? Explain its steps.

IMPORTANT

Definition: Data cleaning is the process of fixing errors and inconsistencies in raw data so the model learns from correct data.

It is always the first step of any machine learning pipeline, because dirty data gives unreliable models.

Step 1: remove duplicate rows, which would otherwise give extra weight to repeated records.

Step 2: fix data types, converting values like text numbers into real numbers.

Step 3: correct invalid entries, such as negative ages or impossible dates.

Step 4: handle outliers, the extreme values that can distort the model.

Step 5: standardise text, making entries like Male, male and M into one form.

Clean data trains faster, gives higher accuracy and produces results that can be trusted.

15 List common data quality problems and how to fix them.

IMPORTANT

Data quality problems are the common defects found in raw real-world data, and each has a standard fix.

Duplicate rows: the same record appears many times; fix by dropping duplicates.

Wrong data types: age stored as text; fix by converting to the correct type.

Outliers: impossible or extreme values like age 250; fix by removing or clipping them.

Inconsistent text: Male, male and M meaning the same thing; fix by converting to one standard form.

Missing values: blank cells; fix by imputation or careful dropping.

Spelling mistakes and extra spaces: fix with stripping and standardisation functions.

Finding and fixing these issues is what data cleaning is all about.

16 Why is feature scaling needed in machine learning?

IMPORTANT

Definition: Feature scaling brings all features into a similar numeric range before training.

It is needed because features often live on very different scales, like age 0 to 100 and salary 0 to 1000000.

Distance-based models measure gaps between points, so the large-range feature dominates the distance completely.

Example: in KNN, salary differences of lakhs would drown out age differences of tens, making age nearly useless.

Gradient-based models also train slowly and unstably when features have very different scales.

Scaling puts every feature on a similar scale, so each feature gets a fair weight in the model.

The two standard methods are normalization (min-max to 0-1) and standardization (z-score).

In short, scaling stops big numbers from bullying small numbers inside the model.

17 Which models need scaling and which do not? Why?

IMPORTANT

Models that need scaling are those based on distances or gradients: KNN, K-means, SVM, neural networks and PCA.

These models compare or combine feature values directly, so unequal ranges distort their maths.

Example: K-means clustering on unscaled age and income would cluster almost entirely on income.

Models that do not need scaling are tree-based: decision trees, random forests and gradient boosting.

Reason: trees split one feature at a time using thresholds, so the range of other features never matters.

A split like age greater than 30 works the same whether salary is in rupees or lakhs.

Still, scaling tree models does no harm; it is simply unnecessary.

In short, scale for distance and gradient models, skip it for trees.

18 Distinguish between normalization and standardization.

IMPORTANT

Both are feature scaling methods, but they rescale data in different ways.

Normalization, or min-max scaling, squeezes every value into a fixed range, usually 0 to 1.

It is done by subtracting the minimum and dividing by the range, so the smallest value becomes 0 and the largest becomes 1.

Standardization, or z-score scaling, centers values around mean 0 with standard deviation 1.

It is done by subtracting the mean and dividing by the standard deviation, so values show how many deviations they sit from the mean.

Normalization is bounded between 0 and 1, while standardization is unbounded and handles outliers more gracefully.

Use normalization when you need a fixed range, like for neural networks; use standardization when data is roughly normal or has outliers.

In short, normalization fixes the range, standardization fixes the distribution shape.

19 What are the formulas for min-max normalization and z-score standardization?

IMPORTANT

Min-max normalization rescales a value x into the 0 to 1 range using the column minimum and maximum.

Formula: x_new = (x - min) / (max - min).

Example: a mark of 80 with min 0 and max 100 becomes (80 - 0) / (100 - 0) = 0.8.

Every value in the column is transformed the same way, so order and relative gaps are preserved.

Z-score standardization converts x into standard deviations away from the mean.

Formula: z = (x - mean) / standard_deviation.

Example: a value 70 with mean 60 and standard deviation 10 becomes (70 - 60) / 10 = 1.0, meaning one deviation above average.

After standardization the column has mean 0 and standard deviation 1.

20 What is data transformation? Give an example.

IMPORTANT

Definition: Data transformation changes the shape of a feature distribution to make it more suitable for modeling.

It is used when data is skewed, meaning most values pile on one side with a long tail.

The log transform is the most common example: it compresses large values more than small ones.

Example: income data with a few very high earners becomes much more balanced after applying log.

Other transforms include square root for mild skew and square for left-skewed data.

Transformation helps models that assume roughly normal data, like linear regression.

It is usually applied after checking a histogram of the feature.

In short, transformation reshapes the data so the model sees a cleaner pattern.

MODULE 2

Numerical, Categorical, Selection

2

EXAM PATTERN (from real MGU question papers)

FE theory exam: 35 marks, 1 hour (MG3DSEBCA201 Honours model paper, mgu.ac.in Oct 2025).

Printed: 10 x Part A (2 marks) + 5 x Part B (5 marks) = 45 marks printed, so there IS choice.

Likely: Part A all compulsory (20) + answer any 3 of 5 in Part B (15) = 35. Exact choice rule UNVERIFIED - confirm from the paper.

Module 2 gave 4 of 10 Part A + 3 of 5 Part B (5-mark: RFE, extraction vs selection, interaction features) in the model paper.

Strongest cross-paper repeats: feature selection methods (3x), dimensionality reduction (3x), PCA steps (2x), extraction vs selection (2x).

1 What is binning? Explain with an example.

IMPORTANT

Definition: Binning is the process of grouping continuous numeric values into a small number of fixed intervals called bins.

Each bin gets a label, turning numbers into categories the model can handle easily.

Example: ages from 0 to 100 are grouped as 0-18 Child, 19-35 Youth, 36-60 Adult and 61+ Senior.

A customer aged 25 falls in the Youth bin, and the model uses the bin label instead of the exact age.

Binning reduces the effect of small noise, since ages 24 and 26 are treated the same.

It also helps models find patterns inside ranges, like young customers buy more.

The two main methods are equal-width bins with the same interval size and equal-frequency bins with the same record count.

In short, binning trades fine detail for stable, meaningful groups.

2 Distinguish between equal-width and equal-frequency binning.

IMPORTANT

Both are binning methods, but they divide the data using different rules.

Equal-width binning splits the value range into bins of the same size, like 0-25, 25-50, 50-75 and 75-100.

It is simple and keeps the ranges easy to explain, but skewed data can leave some bins nearly empty.

Equal-frequency binning puts the same number of records in each bin, so bin ranges stretch where data is sparse.

It handles skewed data better because every bin gets a fair share of records.

Example: for income data, equal-width bins may pile everyone into the first bin, while equal-frequency bins spread them evenly.

In pandas, cut() makes equal-width bins and qcut() makes equal-frequency bins.

In short, use equal-width for simple even data and equal-frequency for skewed data.

3 What is discretization? How is it related to binning?

IMPORTANT

Definition: Discretization is the general process of converting continuous data into discrete groups or categories.

Continuous values like 72.4 marks become labels like First Class, which models handle more easily.

Binning is the most common discretization method: it cuts the range into intervals called bins.

So binning is one way of doing discretization, just as cut() is one way of binning in pandas.

Other discretization ideas include grouping by business rules, like tax slabs defined by the government.

The goal is always the same: reduce noise and make patterns clearer.

Discretization is widely used before models that prefer categories, like decision trees and naive Bayes.

In short, discretization is the concept and binning is its most popular technique.

4 What are polynomial features? Why are they used?

IMPORTANT

Definition: Polynomial features are new columns made from powers of existing features, like x^2 and x^3.

They are used because simple linear models can only draw straight lines, but real data often curves.

Adding x^2 lets a linear model fit a curve while staying a linear model in the new features.

Example: house price may grow faster than size, so adding size-squared captures that curve.

In sklearn, PolynomialFeatures(degree=2) creates these columns automatically.

They are also useful when the true relationship is known to be curved, like projectile motion.

The danger is overfitting: too many powers make the model memorise noise instead of the trend.

In short, polynomial features give simple models the power to bend.

5 What is an interaction feature? Give an example.

IMPORTANT

Definition: An interaction feature combines two or more features, usually by multiplying them, to capture their joint effect.

Some effects appear only when features act together, and single features miss them completely.

Example: rooms x area captures house size better than rooms and area separately.

A 3-room flat of 500 sq ft and a 3-room flat of 1500 sq ft are very different, and only the product shows it.

Another example: in marketing, age group x income level can reveal that young high-earners behave uniquely.

Interaction features are created manually by multiplication or automatically by PolynomialFeatures with interaction_only=True.

Like polynomial features, they must be used carefully to avoid overfitting.

In short, interaction features let the model see teamwork between columns.

6 What precautions should be taken while creating polynomial features?

IMPORTANT

Polynomial features are powerful, but careless use quickly damages the model.

Precaution 1: create them only when the relationship actually looks curved; adding powers to straight-line data adds pure noise.

Precaution 2: keep the degree low, usually 2 or 3, because higher degrees overfit very fast.

Precaution 3: always scale features first, since x^2 of an unscaled large number explodes in size.

Precaution 4: watch the feature count; degree 3 on 10 features creates hundreds of columns.

Precaution 5: use regularization like Ridge or Lasso to control the extra flexibility.

Precaution 6: validate on test data to confirm the curves reflect reality, not noise.

In short, use low degrees, scale first, and always check against overfitting.

7 What is one-hot encoding? Explain with an example.

IMPORTANT

Definition: One-hot encoding converts each category of a column into its own new column containing only 0 or 1.

For every row, the column of its own category gets 1 and all other new columns get 0.

Example: a Colour column with Red, Green, Blue becomes three columns: Colour_Red, Colour_Green, Colour_Blue.

A red shirt becomes 1, 0, 0; a green shirt becomes 0, 1, 0; a blue shirt becomes 0, 0, 1.

This is why it is called one-hot: exactly one column is hot (1) per row.

It is best for nominal data with few categories and no natural order, like gender or city.

It avoids the false ordering problem of label encoding, since no category looks greater than another.

The price is extra columns, so it suits columns with a small number of categories.

8 What is the main disadvantage of one-hot encoding?

IMPORTANT

The main disadvantage is the explosion of columns when a category has many unique values.

A column with 1000 cities becomes 1000 new columns, most of them zeros.

This wastes memory and slows down training considerably.

It also makes the data very sparse, which weakens many models.

This problem is known as the curse of dimensionality.

One common fix is to drop one column to avoid the dummy variable trap in linear models.

Better fixes for high-cardinality data are hashing or target encoding.

In short, one-hot encoding is excellent for few categories but breaks down for many.

9 Write code for one-hot encoding in pandas and sklearn.

IMPORTANT

One-hot encoding is a one-line operation in both pandas and sklearn.

In pandas, get_dummies() converts the chosen columns instantly:

```
df = pd.get_dummies(df, columns=["city"])
```

This replaces the city column with one 0/1 column per city.

In sklearn, OneHotEncoder fits into ML pipelines:

```
from sklearn.preprocessing import OneHotEncoder
enc = OneHotEncoder(sparse_output=False)
X_new = enc.fit_transform(df[["city"]])
```

The sklearn version remembers the categories, so it encodes new data consistently during prediction.

Use pandas for quick analysis and sklearn when building a proper training pipeline.

10 What is label encoding? Write the sklearn code.

IMPORTANT

Definition: Label encoding replaces each category with an integer label, keeping everything in a single column.

Example: sizes S, M, L become 0, 1, 2.

It is simple, compact and needs no extra columns.

The sklearn code is:

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["size"] = le.fit_transform(df["size"])
```

fit_transform learns the mapping on training data; transform() alone is used on new data.

The original labels can be recovered with le.inverse_transform() when needed.

In short, label encoding is the compact way to number categories.

11 When should label encoding be used, and when avoided?

IMPORTANT

Use label encoding for ordinal data, where the order of categories is real and meaningful.

Example: grades Poor=0, Average=1, Good=2, where the numbers correctly reflect increasing quality.

Tree-based models also tolerate label-encoded nominal data fairly well, since they split on thresholds.

Avoid it for nominal data with linear models, because the numbers create a false order.

Example: encoding cities as 1, 2, 3 tricks a linear model into treating city 3 as greater than city 1.

In such cases one-hot encoding is the safer choice.

The decision always depends on both the data type and the model being used.

In short, use label encoding when order is real, avoid it when order would be invented.

12 Compare one-hot encoding and label encoding.

IMPORTANT

Both convert categories to numbers, but they do it in opposite ways.

One-hot encoding creates many columns with 0/1 values: one new column per category.

Label encoding creates a single column with integer values, one number per category.

One-hot avoids false ordering completely, since no category looks larger than another.

Label encoding is compact and memory-friendly, and it preserves real order in ordinal data.

Example: Gender M/F suits one-hot encoding; Grade A/B/C suits label encoding.

One-hot struggles with many categories due to column explosion; label encoding struggles with nominal data due to false order.

In short, one-hot is safe but wide, label encoding is compact but order-sensitive.

13 **A 'city' column has 500 cities. Which encoding would you choose and why?**

IMPORTANT

A city column with 500 unique values is called high-cardinality data, and it needs special care.

One-hot encoding would create 500 new columns, which wastes memory and slows training badly.

Label encoding keeps just one column, which is far more practical here.

Tree-based models like random forests handle label-encoded cities reasonably well.

For linear models, even better options are hashing encoding or target encoding, which compress the cities smartly.

The choice also depends on the model: trees tolerate label encoding, linear models prefer smarter encodings.

Blindly one-hot encoding 500 cities is one of the most common beginner mistakes.

In short, avoid one-hot for 500 cities; use label encoding for trees or target/hashing encoding for linear models.

14 **Distinguish between feature extraction and feature selection. (FE model paper; MCA ML Oct 2025)**

REPEATED

Both reduce the number of features, but they do it in fundamentally different ways.

Feature selection chooses a subset of the original features and drops the rest.

Example: keeping only age and income from 20 columns and deleting the other 18.

Feature extraction builds brand-new features by combining the old ones.

Example: PCA merges 20 columns into 3 new components that capture most of the information.

Selection keeps the original column names and meanings, so results stay easy to explain.

Extraction usually compresses better when features are highly correlated, but the new features have no simple meaning.

In short, selection picks the best old features, extraction invents new compact ones.

15 When would you prefer feature selection over extraction?

IMPORTANT

Prefer feature selection when interpretability matters, because the original column names are kept.

Example: a bank must explain why a loan was rejected, so it needs real features like income and credit score, not PCA components.

Selection is also simpler to implement and easier to debug.

It works well when a few features are clearly useful and the rest are noise.

Prefer extraction when maximum compression is the goal and meaning does not matter.

Example: compressing 1000 image pixels into 50 components for faster training.

Extraction also helps when features are highly correlated and selection cannot separate them cleanly.

In short, choose selection for explainable results and extraction for maximum compression.

16 Explain the steps involved in feature selection.

IMPORTANT

Feature selection follows a systematic search process in four stages.

Stage 1, subset generation: create candidate feature subsets to try, like {age}, {age, income} and {age, income, city}.

Stage 2, subset evaluation: score each subset using a statistical measure or the actual model accuracy.

Stage 3, stopping criterion: halt the search when scores stop improving or a time limit is reached.

Stage 4, validation: confirm the chosen subset performs well on fresh unseen test data.

This process can be forward (adding features), backward (removing features) or exhaustive for small data.

Skipping validation is dangerous, because a subset can look good on training data but fail on new data.

In short, generate, evaluate, stop sensibly and validate.

17 What are the four stages of the feature selection process?

IMPORTANT

The four standard stages of feature selection are generation, evaluation, stopping and validation.

Generation: create candidate subsets of features to examine.

Evaluation: score each subset with a metric, a statistical test or model performance.

Stopping: halt the search when the score stops improving, a target size is reached, or time runs out.

Validation: test the final selected subset on unseen data to confirm it truly works.

These stages apply whether the method is filter, wrapper or hybrid.

Missing the stopping stage wastes time; missing validation risks overfitting.

In short, the four stages are generate, evaluate, stop and validate.

18 Why is a stopping criterion needed in feature selection?

IMPORTANT

A stopping criterion tells the search when to halt, and without it the process has no natural end.

With n features there are 2^n possible subsets, so an exhaustive search quickly becomes impossible.

Example: 30 features give over a billion subsets, which no computer can try fully.

A good stopping rule saves huge amounts of time by quitting once scores stop improving.

Example: stop when accuracy does not improve for 3 rounds in a row.

It also prevents overfitting, because endlessly chasing tiny training gains usually hurts new data.

Common rules are a fixed number of features, a score threshold or a time budget.

In short, the stopping criterion keeps the search fast, practical and safe.

19 **Explain filter, wrapper and hybrid feature selection methods. (IMCA DS Dec 2025, asked twice; MCA ML Oct 2025)**

SURE MARK

These are the three families of feature selection, differing in how they judge features.

Filter methods rank features using statistics alone, with no model involved.

Example: keep features with high correlation to the target, or use the chi-square test for categories.

They are very fast and work well on large data, but they may miss interactions between features.

Wrapper methods train the actual model on many feature subsets and keep the best-scoring subset.

Example: recursive feature elimination (RFE) repeatedly drops the weakest feature.

They are accurate because they test with the real model, but they are slow on large data.

Hybrid or embedded methods perform selection inside training itself; example: Lasso regression shrinks useless feature weights to zero, and random forests rank features by importance.

20 **Compare filter and wrapper methods with examples.**

IMPORTANT

Filter and wrapper methods differ in speed, accuracy and how they use the model.

Filter methods use only statistics, so they are fast and model-independent.

Example: a chi-square filter quickly ranks 1000 gene columns without training anything.

But filters judge features one by one and can miss useful combinations.

Wrapper methods train the real model on each candidate subset, so they are highly accurate.

Example: RFE with logistic regression on 20 columns finds the subset that truly performs best.

The price is speed: wrappers are far too slow for thousands of features.

In short, use filters for speed on big data and wrappers for accuracy on small data.

21 What is forward selection and backward elimination?

IMPORTANT

Both are wrapper-style search strategies that build the feature set step by step.

Forward selection starts with an empty set and adds the best feature one at a time.

After each addition the model is scored, and the process stops when the score stops improving.

Backward elimination works in reverse: it starts with all features and removes the weakest one at a time.

It also stops when removing more features starts hurting the score.

Forward selection is cheaper when the final set is small; backward elimination sees feature interactions better at the start.

Both are greedy methods, meaning they never reconsider an earlier choice.

In short, forward selection builds up and backward elimination trims down.

22 What are embedded methods? Give examples.

IMPORTANT

Definition: Embedded methods perform feature selection as a natural part of model training.

Unlike filters and wrappers, there is no separate selection step; the model selects while it learns.

Example 1: Lasso regression adds a penalty that forces the weights of useless features to exactly zero.

The surviving non-zero weights reveal the selected features automatically.

Example 2: random forests rank features by importance during training, so the lowest-ranked ones can be dropped.

Embedded methods are faster than wrappers because selection and training happen together.

They are also more accurate than filters because the real model guides the choice.

In short, embedded methods select features as a side effect of training.

23 **Explain how Recursive Feature Elimination (RFE) works as a wrapper method for feature selection.**

IMPORTANT

Definition: RFE is a wrapper method that finds the best features by repeatedly training a model and dropping the weakest feature.

It is called a wrapper because it wraps around an actual machine learning model to judge features.

Step 1: train the model on all features and rank them by importance, like the weights in logistic regression.

Step 2: remove the single weakest feature from the set.

Step 3: retrain the model on the remaining features and rank again.

These steps repeat until only the desired number of features is left.

Example: from 20 columns, RFE with logistic regression can narrow down to the 5 columns that truly drive predictions.

It is accurate because the real model judges every subset, but slow on large data since it retrains many times.

In short, RFE eliminates features one by one until only the strongest survive.

24 **What is dimensionality reduction? Why is it important? (IMCA DS Dec 2025; MCA ML Dec 2023, asked twice)**

SURE MARK

Definition: Dimensionality reduction is the process of reducing the number of features in a dataset while keeping as much information as possible.

High-dimensional data, like images with thousands of pixels, is slow to process and hard to visualise.

It is important first for speed: fewer features mean faster training and less memory.

Second, it fights overfitting: extra useless dimensions give the model more noise to memorise.

Third, it enables visualisation: reducing to 2 or 3 dimensions lets us actually plot and see the data.

Two main routes exist: feature selection, which picks the best original features, and feature extraction like PCA, which builds new compact ones.

Example: 100 correlated sensor readings compressed to 5 PCA components keep nearly all the useful signal.

In short, dimensionality reduction keeps the signal, drops the bulk, and makes models faster

and clearer.

25 What is PCA? Explain its steps. (IMCA DS Dec 2025; MCA ML Oct 2025)

REPEATED

Definition: PCA (Principal Component Analysis) is a feature extraction technique that reduces dimensions by finding new uncorrelated axes called principal components.

The first component captures the maximum variance (spread) in the data, the second captures the next most, and so on.

It is used for visualization, noise reduction and faster model training.

Step 1: standardize all features so that large-range features do not dominate.

Step 2: compute the covariance matrix, which shows how features vary together.

Step 3: find its eigenvectors and eigenvalues; the eigenvectors are the new axes and eigenvalues show their importance.

Step 4: keep the top components that capture most of the variance and project the data onto them.

Example: 100 correlated sensor readings can be compressed into 5 components with little information loss.

26 Why is standardization needed before PCA?

IMPORTANT

PCA looks for the directions of maximum variance, so scale differences would hijack the result.

A feature with a huge range, like salary in lakhs, has far more variance than age in tens, purely because of units.

Without standardization, the first principal component would simply point along salary and ignore everything else.

Standardization puts every feature on the same scale, with mean 0 and standard deviation 1.

After that, variance truly reflects information content, not measurement units.

Example: without scaling, salary crushes age in the components; with scaling, both contribute fairly.

This is why every PCA pipeline starts with StandardScaler.

In short, standardization stops big units from bullying small units in PCA.

27 What are principal components?

IMPORTANT

Definition: Principal components are the new axes created by PCA from combinations of the original features.

Each component is a weighted mix of all original features, pointing in a direction of high data spread.

They are uncorrelated with each other, meaning each one captures fresh information the others missed.

The first component holds the maximum possible variance, the second holds the next most, and so on.

Components are ordered by their eigenvalues, which measure how much variance each one explains.

Example: from height and weight, the first component might represent overall body size.

We usually keep only the top few components and discard the rest as noise.

In short, principal components are new smarter axes ranked by importance.

28 List three applications of PCA.

IMPORTANT

PCA is used wherever high dimensions cause trouble, and three classic applications stand out.

Application 1: visualization. Reducing 100 features to 2 lets us plot and actually see clusters in the data.

Application 2: noise reduction and faster training. Dropping low-variance components removes noise and shrinks the data models must process.

Application 3: compression. Images, sensor readings and gene data are compressed into fewer components before modeling.

It is also used for anomaly detection, since points far from the main components look suspicious.

In all cases the idea is the same: keep the signal, drop the excess dimensions.

PCA is often the first step tried when a dataset feels too wide to handle.

In short, PCA visualizes, denoises and compresses high-dimensional data.

29 Write sklearn code for PCA with 2 components.

IMPORTANT

PCA in sklearn takes just a few lines, but standardization must come first.

Code:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
X_scaled = StandardScaler().fit_transform(X)
X_pca = PCA(n_components=2).fit_transform(X_scaled)
```

StandardScaler gives every feature mean 0 and standard deviation 1.

PCA(n_components=2) keeps the two most important components.

The result X_pca has only 2 columns and can be plotted or fed into a model.

Remember: fit the scaler and PCA on training data only, then transform test data with the same objects.

In short, scale first, then reduce to 2 components.

MODULE 3

Lab Practice

3

EXAM PATTERN (from real MGU question papers)

FE theory exam: 35 marks, 1 hour (MG3DSEBCA201 Honours model paper, mgu.ac.in Oct 2025).

Printed: 10 x Part A (2 marks) + 5 x Part B (5 marks) = 45 marks printed, so there IS choice.

Likely: Part A all compulsory (20) + answer any 3 of 5 in Part B (15) = 35. Exact choice rule UNVERIFIED - confirm from the paper.

Module 3 (lab: numpy, pandas, sklearn, EDA, log transform, text) has ZERO questions in the model paper and none in related papers.

M3 is almost certainly assessed through the PRACTICAL exam, not theory. Study it for practicals; theory marks come from M1 and M2.

1 List the Python libraries used in ML and their purposes.

IMPORTANT

Python dominates ML because its libraries cover every step of the pipeline.

numpy provides fast numerical arrays and maths operations; it is the foundation under almost everything.

pandas provides DataFrames for loading, cleaning and exploring tabular data with simple commands.

sklearn offers ready-made ML models plus preprocessing tools like scalers, encoders and metrics.

matplotlib is the basic plotting library, while seaborn builds prettier statistical plots on top of it.

nltk handles text processing with tokenizers, stopword lists and stemmers.

Kaggle is not a library but the main source of ready datasets for practice.

In short, numpy computes, pandas organises, sklearn models, and matplotlib shows.

2 What is the difference between matplotlib and seaborn?

IMPORTANT

Both draw plots in Python, but they sit at different levels of convenience.

matplotlib is the basic low-level plotting library; you control every detail, but simple plots need several lines.

seaborn is built on top of matplotlib and is designed for statistical plots with beautiful defaults.

Example: sns.heatmap() draws a full correlation matrix heatmap in a single line.

seaborn understands pandas DataFrames directly, so plotting a column is effortless.

Use matplotlib when you need total custom control; use seaborn for quick attractive exploratory

plots.

Most data scientists use both: seaborn for speed, matplotlib for fine-tuning.

In short, seaborn is matplotlib with better looks and less effort.

3 What is nltk used for in feature engineering?

IMPORTANT

nltk, the Natural Language Toolkit, is the standard Python library for text preprocessing.

It provides tokenizers that split sentences into words.

It provides stopword lists for removing common words like the and is.

It provides stemmers and lemmatizers that reduce words to their base forms.

Example: a product review is tokenized, cleaned of stopwords and stemmed before being turned into features.

These cleaned words then become features through methods like bag-of-words or TF-IDF.

Without nltk-style cleaning, text features stay noisy and models perform poorly.

In short, nltk turns messy raw text into clean tokens ready for feature creation.

4 How do you find and count missing values in pandas?

IMPORTANT

The first step of handling missing data is finding where the gaps are.

isnull() returns a True/False table marking every missing cell as True.

Chaining sum() counts the True values, giving the missing count per column.

Code:

```
df.isnull().sum()
```

This prints one number per column, instantly showing which columns have gaps.

For a quick overall check, df.isnull().sum().sum() gives the total missing cells.

Always run this before deciding whether to fill or drop.

In short, isnull().sum() is the standard missing-value detector in pandas.

5 Write pandas code to fill missing values with mean, median and mode.

IMPORTANT

The fillna() method replaces missing values, and the strategy depends on the column type.

Fill a numeric column with its mean:

```
df["age"] = df["age"].fillna(df["age"].mean())
```

Use the median for skewed numeric data, where the mean would mislead:

```
df["salary"] = df["salary"].fillna(df["salary"].median())
```

Use the mode for categorical columns, picking the most frequent value:

```
df["city"] = df["city"].fillna(df["city"].mode()[0])
```

The [0] is needed because mode() returns a Series even for a single value.

Choose the strategy by data type: mean for normal numbers, median for skewed numbers, mode for categories.

In short, fillna with the right statistic keeps the column honest.

6 When do you use median instead of mean for imputation?

IMPORTANT

Use the median instead of the mean whenever the data is skewed or contains outliers.

The mean is pulled toward extreme values, so it stops representing the typical value.

Example: salaries 30000, 32000, 35000 and 5000000 have a mean near 13 lakhs, which describes nobody.

The median is the middle value, so extreme values cannot drag it around.

In that salary example the median is around 33500, a far more sensible fill value.

Filling with a distorted mean would inject wrong information into every blank cell.

The same logic applies to house prices, incomes and any long-tailed data.

In short, use the median whenever outliers or skew make the mean untrustworthy.

7 Write sklearn code for missing-value imputation.

IMPORTANT

sklearn fills missing values with SimpleImputer, which fits neatly inside ML pipelines.

Unlike pandas fillna, it learns the fill values on training data and reuses them on test data.

Code for mean imputation:

```
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy="mean")
X_filled = imp.fit_transform(X)
```

The strategy can be mean, median, most_frequent or constant.

For a constant fill, add fill_value, like SimpleImputer(strategy="constant", fill_value=0).

Using it in a pipeline prevents data leakage between train and test sets.

In short, SimpleImputer is the pipeline-friendly way to fill gaps.

8 Write pandas code to remove duplicates and fix data types.

IMPORTANT

Cleaning starts with removing duplicates and correcting column types.

Remove duplicate rows so repeated records do not get extra weight:

```
df = df.drop_duplicates()
```

Convert wrongly typed columns, like age stored as text, to the right type:

```
df["age"] = df["age"].astype(int)
```

Clean messy text columns by stripping spaces and unifying case:

```
df["name"] = df["name"].str.strip().str.lower()
```

After stripping, " Male " and "male" become the same value.

Always check dtypes with df.dtypes after conversion to confirm the fix.

These three lines solve the most common cleaning problems in one pass.

9 How do you detect and remove outliers using the IQR method?

IMPORTANT

Definition: The IQR (interquartile range) method finds outliers using the spread of the middle 50 percent of data.

IQR equals Q3 minus Q1, the third quartile minus the first quartile.

Any value below Q1 - 1.5*IQR or above Q3 + 1.5*IQR is treated as an outlier.

Code:

```
q1 = df["age"].quantile(0.25)
q3 = df["age"].quantile(0.75)
iqr = q3 - q1
df = df[(df["age"] >= q1 - 1.5*iqr) & (df["age"] <= q3 + 1.5*iqr)]
```

This keeps only the rows inside the acceptable range.

The IQR method is robust because quartiles, unlike mean and standard deviation, resist outliers themselves.

In short, quartiles define normal, and anything far outside is an outlier.

10 What is EDA? Why is it done before modeling?

IMPORTANT

Definition: EDA (Exploratory Data Analysis) is the process of understanding data with statistics and plots before building a model.

It reveals missing values, outliers, skewed distributions and wrong data types early.

It shows how features relate to each other and to the target, guiding feature selection.

Skipping EDA means the model may silently learn from garbage data.

Example: EDA might reveal that 40 percent of ages are missing, which changes the whole cleaning plan.

It also suggests which features need scaling, encoding or transformation.

The common EDA toolkit is describe(), histograms, boxplots and a correlation heatmap.

In short, EDA is done first because you must understand the data before you can model it.

11 What does a boxplot show? Explain its parts.

IMPORTANT

Definition: A boxplot is a compact visual summary of a numeric column distribution.

The box stretches from Q1 (25th percentile) to Q3 (75th percentile), showing the middle 50 percent of data.

A line inside the box marks the median, the middle value.

The whiskers extend to the normal range of data on both sides.

Dots beyond the whiskers are outliers, values far from the rest.

It quickly reveals spread, skew and extreme values of a column at one glance.

Comparing boxplots across categories, like salary by department, instantly shows group differences.

In short, a boxplot packs quartiles, median and outliers into one small picture.

12 What is a correlation matrix? How is it read?

IMPORTANT

Definition: A correlation matrix is a table showing the correlation value between every pair of features.

Each cell holds a number between -1 and +1 measuring how two features move together.

Values near +1 mean strong positive relation: both rise together, like study hours and marks.

Values near -1 mean strong negative relation: one rises as the other falls.

Values near 0 mean weak or no linear relation.

It is read as a heatmap, where dark or intense cells highlight the strongest relations.

It helps drop redundant features: two features with 0.95 correlation carry nearly the same information.

In short, the correlation matrix maps the relationships hiding in the data.

13 Write code for a histogram, boxplot and correlation heatmap.

IMPORTANT

These three plots form the core visual toolkit of EDA.

Histogram of one column shows its distribution shape:

```
df["age"].hist()
```

Boxplot of one column shows median, quartiles and outliers:

```
df.boxplot(column="age")
```

Correlation heatmap of all numeric columns shows every pairwise relation:

```
import seaborn as sns
sns.heatmap(df.corr(), annot=True)
```

The annot=True flag prints the actual correlation numbers inside the cells.

Run these three right after loading any new dataset.

In short, hist for shape, boxplot for spread, heatmap for relations.

14 List the steps of EDA in order.

IMPORTANT

EDA follows a fixed order so that nothing important is missed.

Step 1: inspect the basics with shape, info() and describe() to see size, types and summary stats.

Step 2: find missing values with isnull().sum() and decide how to handle them.

Step 3: plot distributions with histograms and boxplots to see shape, skew and outliers.

Step 4: check relationships with a correlation heatmap and scatter plots against the target.

Step 5: note every issue found: what needs cleaning, scaling, encoding or transformation.

Only after these steps should modeling begin.

In short: inspect, find gaps, plot shapes, check relations, then plan the fixes.

15 Write pandas code for equal-width and equal-frequency binning.

IMPORTANT

pandas offers two one-line functions for binning.

cut() makes equal-width bins with the same interval size:

```
df["age_group"] = pd.cut(df["age"], bins=4)
```

This splits the age range into 4 equal intervals automatically.

qcut() makes equal-frequency bins with the same record count each:

```
df["age_group"] = pd.qcut(df["age"], q=4)
```

Custom ranges and meaningful labels can be supplied to cut():

```
pd.cut(df["age"], bins=[0, 18, 35, 60, 100], labels=["Child", "Youth", "Adult", "Senior"])
```

After binning, check value_counts() to confirm the split looks sensible.

In short, cut() for fixed ranges and qcut() for balanced groups.

16 When would you use cut() instead of qcut()?

IMPORTANT

Use cut() when the bin ranges themselves carry meaning and should stay fixed.

Example: age groups 0-18, 18-35, 35-60 make business sense regardless of data spread.

Use qcut() when you want balanced groups, especially with skewed data.

Example: income is heavily skewed, so qcut() gives four groups with equal customer counts.

cut() on skewed data can leave some bins nearly empty, wasting the grouping.

qcut() on uniform data can create odd-looking ranges, since it forces equal counts.

The choice depends on whether meaningful ranges or balanced groups matter more.

In short, cut() for meaningful fixed ranges and qcut() for balanced skewed data.

17 Why are labels given to bins? Give an example.

IMPORTANT

Labels turn numeric bin ranges into meaningful names that humans understand.

Raw bins like (18, 35] mean little in a report, but the label Youth is instantly clear.

Example: bins [0, 18, 60, 100] labeled Child, Adult, Senior.

Labeled bins make plots, tables and presentations far easier to read.

They also make the feature self-documenting for anyone using the model later.

In pandas, labels are passed directly: pd.cut(..., labels=["Child", "Adult", "Senior"]).

The number of labels must match the number of bins exactly.

In short, labels translate bin math into human language.

18 Write sklearn code to create polynomial features of degree 2.

IMPORTANT

PolynomialFeatures generates polynomial and interaction terms automatically.

Code:

```
from sklearn.preprocessing import PolynomialFeatures
poly = PolynomialFeatures(degree=2, include_bias=False)
X_new = poly.fit_transform(X)
```

On features [x1, x2] this creates x1, x2, x1^2, x2^2 and x1*x2.

include_bias=False skips the constant column of ones, which most models add themselves.

Always scale features before this step, since squared large values explode in size.

Use poly.get_feature_names_out() to see the names of the generated columns.

In short, three lines turn straight-line features into curve-ready ones.

19 How do you create only interaction features in sklearn?

IMPORTANT

Sometimes you want feature combinations without the squared terms.

PolynomialFeatures has a dedicated mode for exactly this.

Code:

```
poly = PolynomialFeatures(degree=2, interaction_only=True, include_bias=False)
X_new = poly.fit_transform(X)
```

On [x1, x2, x3] this creates only x1*x2, x1*x3 and x2*x3, skipping x1^2 and friends.

This is useful when you suspect teamwork between features but not curved single-feature effects.

It also keeps the feature count lower than full polynomial expansion.

In short, interaction_only=True gives products without powers.

20 What is skewed data? How does log transformation fix it?

IMPORTANT

Definition: Skewed data piles up on one side with a long tail stretching to the other, like incomes with a few very high earners.

Most values sit in a small region while the tail holds extreme values.

This hurts models that assume roughly balanced data, because the tail dominates.

Log transformation compresses large values much more than small ones.

After the transform, the long tail shrinks and the distribution looks far more normal.

Example: incomes 30000, 35000, 40000 and 5000000 become much more comparable on the log scale.

The model can then learn the real pattern instead of chasing the tail.

In short, log transform tames the tail and balances skewed data.

21 Why is log1p preferred over log? Write the code.

IMPORTANT

The plain log function has a fatal weakness: log(0) is undefined and crashes the code.

Real data often contains zeros, like zero purchases or zero rainfall days.

log1p computes log(1+x) instead, so zero safely becomes log(1) = 0.

For positive values it behaves almost exactly like log, just shifted.

Code:

```
import numpy as np
df["log_income"] = np.log1p(df["income"])
```

This one line safely transforms the whole column, zeros included.

In short, log1p is log without the zero crash.

22 Write code to apply log transformation and plot before/after.

IMPORTANT

Always verify a transform visually by comparing before and after plots.

Code:

```
import numpy as np
df["income"].hist()
np.log1p(df["income"]).hist()
```

The first plot shows the original skewed shape with its long tail.

The second plot should look more bell-shaped and balanced.

If the tail is still long, a stronger approach like Box-Cox may be needed.

This before/after check takes seconds and confirms the transform actually helped.

In short, plot, transform, plot again.

23 Explain the steps of text preprocessing.

IMPORTANT

Definition: Text preprocessing cleans raw text so it can become reliable features.

Step 1: lowercase everything, so Good and good count as the same word.

Step 2: remove punctuation, numbers and extra spaces that carry no meaning.

Step 3: tokenize, splitting the text into individual words.

Step 4: remove stopwords, the very common words like the and is.

Step 5: stem or lemmatize the remaining words to their base forms.

After these steps the text is a clean list of meaningful tokens.

These tokens then become features through bag-of-words or TF-IDF.

24 Distinguish between stemming and lemmatization.

IMPORTANT

Both reduce words to base forms, but they differ in method and quality.

Stemming chops word endings using fixed rules, which is fast but crude.

Example: studies becomes studi, which is not even a real word.

Lemmatization uses a dictionary and grammar rules to find the true base form.

Example: studies becomes study and better becomes good.

Lemmatization is slower but far more accurate than stemming.

Use stemming for speed on huge data and lemmatization when quality matters.

In short, stemming cuts blindly while lemmatization thinks.

25 What are stopwords? Why are they removed?

IMPORTANT

Definition: Stopwords are very common words like the, is, and and in.

They appear in almost every document and carry little distinctive meaning.

Keeping them wastes memory and slows down feature creation.

Worse, they dilute the signal: a spam filter learns nothing from the word the.

Removing them shrinks the vocabulary and helps the model focus on important words.

nltk provides ready stopword lists for many languages.

Some tasks keep negations like not, since not good means the opposite of good.

In short, stopwords are noise words, and removing them sharpens the text signal.

26 Write nltk code for tokenization and stopword removal.

IMPORTANT

nltk makes tokenization and stopword removal a few lines of code.

Code:

```
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
words = word_tokenize(text.lower())
stops = set(stopwords.words("english"))
clean = [w for w in words if w.isalpha() and w not in stops]
```

word_tokenize splits the text into words; lowercasing unifies the case.

The list comprehension keeps only alphabetic words that are not stopwords.

Note: nltk data (punkt, stopwords) must be downloaded once with nltk.download().

In short, tokenize, filter, and clean text is ready.

27 Write complete sklearn code for PCA keeping 95% variance.

IMPORTANT

This is the standard PCA pipeline: scale first, then reduce.

Code:

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
X_scaled = StandardScaler().fit_transform(X)
X_pca = PCA(n_components=0.95).fit_transform(X_scaled)
```

StandardScaler gives every feature mean 0 and standard deviation 1, which PCA requires.

Passing 0.95 as n_components tells sklearn to keep enough components for 95 percent of variance.

The result X_pca has far fewer columns but keeps 95 percent of the information.

Fit the scaler and PCA on training data only, then transform test data with the same fitted objects.

In short, scale first and let sklearn pick the component count for 95 percent variance.

28 How do you decide how many components to keep in PCA?

IMPORTANT

The number of components is chosen by how much variance you want to keep.

Common targets are 90 or 95 percent of the total variance.

Fit PCA with all components first and inspect explained_variance_ratio_.

Code:

```
pca = PCA().fit(X_scaled)
print(pca.explained_variance_ratio_)
```

Add up the ratios from the largest down until the total crosses your target.

Example: ratios [0.6, 0.25, 0.1, 0.05] mean 2 components keep 85 percent and 3 keep 95 percent.

A scree plot of these ratios shows an elbow where extra components add little.

In short, keep adding components until the kept variance reaches your target.

29 What is explained_variance_ratio_ in sklearn PCA?

IMPORTANT

Definition: explained_variance_ratio_ is an array showing the fraction of total variance held by each principal component.

After fitting PCA, each component gets one number in this array.

Example: [0.6, 0.25, 0.1, 0.05] means the first component holds 60 percent of the information and the second 25 percent.

The values always add up to 1.0 across all components.

It is the main tool for deciding how many components to keep: sum the largest ratios until you reach 90 or 95 percent.

It also reveals the data structure: a steep drop means a few directions dominate.

You can read it right after pca.fit(X_scaled) with no extra computation.

In short, it tells you how much each component is worth keeping.